

Angular

How Web application works frontend server backend server database explain with diagram

How a Web Application Works (Frontend → Server → Backend → Database)

Think of a food delivery app like Swiggy or Zomato.

Components

1. Frontend (UI)

- Angular, React, Vue, HTML/CSS/JavaScript
- Runs in the user's browser
- Shows screens, buttons, forms

2. Web Server / Frontend Server

- Nginx, Apache, CDN
- Serves Angular/React files to the browser
- Handles static content

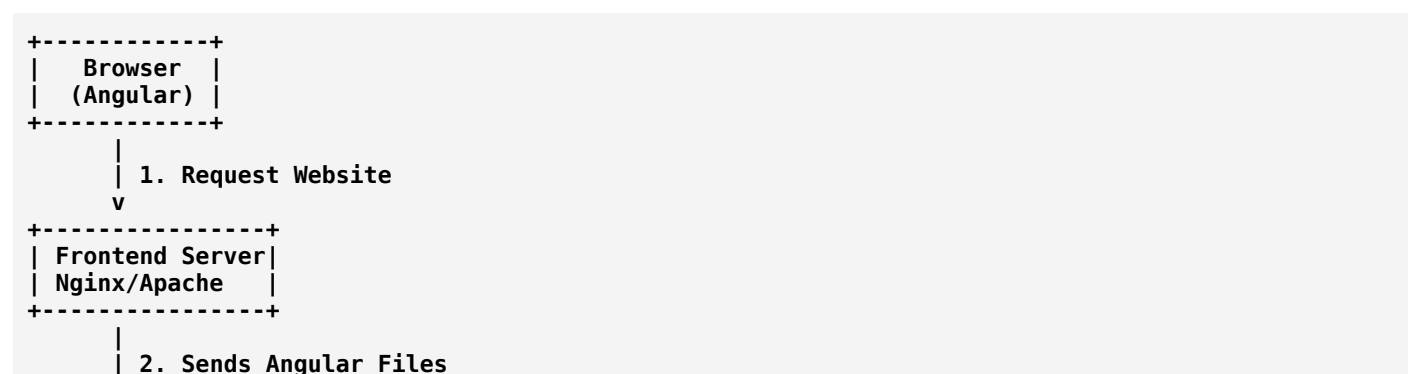
3. Backend Server

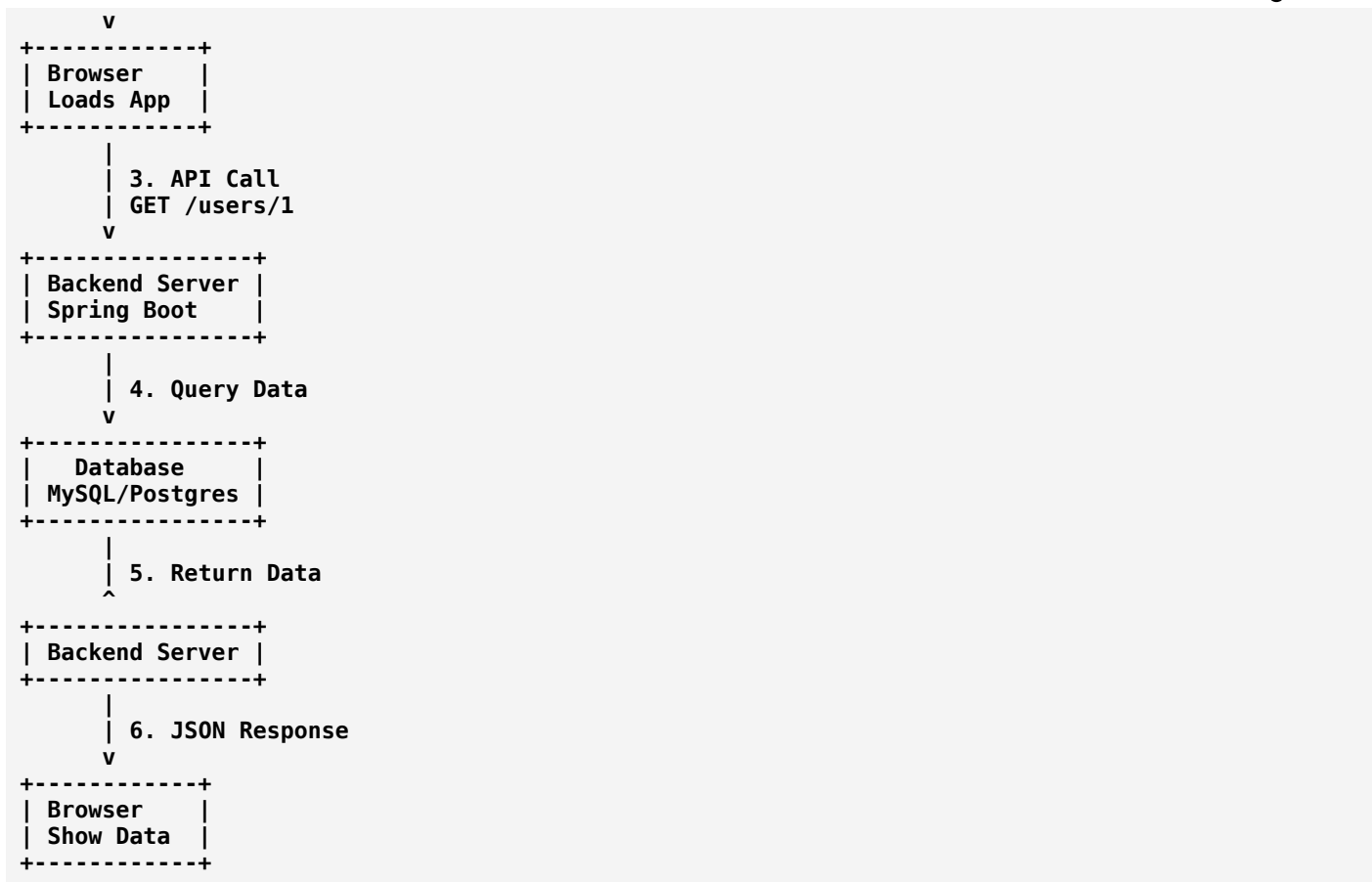
- Java Spring Boot, Node.js, .NET, Python
- Contains business logic
- Processes requests and validations

4. Database

- MySQL, PostgreSQL, Oracle, MongoDB
- Stores application data

Basic Flow





Real Example: Login Flow

Step 1: User Enters Credentials

Username: lawrence
Password: *****

Angular sends:

```
http
POST /login
```

Payload:

```
JSON
{
  "username": "lawrence",
  "password": "123456"
}
```

Step 2: Backend Receives Request

Spring Boot Controller:

```
Java
@PostMapping("/login")
public ResponseEntity<?> login(
```

```
@RequestBody LoginRequest request) {  
    return authService.login(request);  
}
```

Step 3: Backend Checks Database

```
SQL  
SELECT *  
FROM users  
WHERE username='lawrence';
```

Step 4: Database Returns User

```
JSON  
{  
  "id":1,  
  "username":"lawrence",  
  "password":"encryptedPassword"  
}
```

Step 5: Backend Validates Password

If correct:

```
Java  
JWT Token Generated
```

Response:

```
JSON  
{  
  "token":"eyJhbGc..."  
}
```

Step 6: Angular Stores Token

```
JavaScript  
localStorage.setItem("token", token);
```

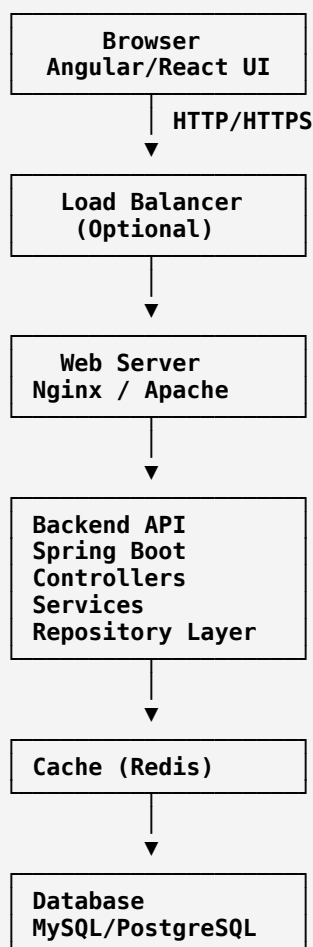
Step 7: User Opens Dashboard

Angular sends:

```
http  
GET /dashboard  
Authorization: Bearer token
```

Backend validates JWT and returns data.

Detailed Architecture



What Happens When User Clicks a Button?

Suppose user clicks "**Get Employees**"

Angular

```
TypeScript
this.http.get("/api/employees")
```

Browser

```
http
GET /api/employees
```

Spring Controller

```
Java
@GetMapping("/employees")
public List<Employee> getEmployees() {
    return employeeService.getEmployees();
}
```

Service Layer

```
Java
public List<Employee> getEmployees() {
    return employeeRepository.findAll();
}
```

Repository Layer

```
Java
employeeRepository.findAll();
```

SQL Executed

```
SQL
SELECT * FROM employee;
```

Database Returns

```
JSON
[
  {
    "id":1,
    "name":"John"
  },
  {
    "id":2,
    "name":"Mike"
  }
]
```

Angular Displays Data

```
HTML
John
Mike
```

Layers in Spring Boot

```
Controller Layer
    ↓
Service Layer
    ↓
Repository Layer
    ↓
Database
```

Controller

Handles HTTP requests.

```
Java
@GetMapping("/users")
```

Service

Business logic.

```
Java
calculateSalary();
validateUser();
```

Repository

Database operations.

```
Java
findById();
save();
delete();
```

Complete Request Lifecycle

```
User
↓
Angular UI
↓
API Call
↓
Controller
↓
Service
↓
Repository
↓
Database
↓
Repository
↓
Service
↓
Controller
↓
JSON Response
↓
Angular
↓
Display on Screen
```

Where Common Technologies Fit

Layer	Technologies
Frontend	Angular, React, Vue
Web Server	Nginx, Apache
Backend	Java Spring Boot
Security	Spring Security, JWT
Cache	Redis
Messaging	Kafka, RabbitMQ
Database	MySQL, PostgreSQL, Oracle
Container	Docker
Orchestration	Kubernetes, OpenShift

Layer	Technologies
CI/CD	Jenkins, GitLab CI
Monitoring	Grafana, Prometheus

Interview Question

Q: Why do we need both Frontend and Backend?

Answer:

- Frontend is responsible for displaying data and taking user input.
- Backend contains business logic, validations, security, and database access.
- Direct database access from frontend would be insecure, so frontend communicates with backend APIs, and backend communicates with the database.

A common interview follow-up is: **"Explain what happens when you type `www.amazon.com` in a browser."** The answer includes DNS lookup → web server response → frontend files download → API calls → backend processing → database queries → response rendering.

What is angular explain features
Angular vs react

What is Angular?

Angular is a TypeScript-based front-end framework developed and maintained by Google for building **Single Page Applications (SPAs)** and large-scale web applications.

It provides everything needed to build a web application:

- UI Components
- Routing
- Forms
- HTTP Calls
- Dependency Injection
- State Management
- Testing Support

Unlike React, Angular is a **complete framework**.

Angular Architecture

```

Components
↓
Templates (HTML)
↓
Services
↓
HTTP Client
↓
Backend APIs

```

Example:

```

LoginComponent
↓
LoginService
↓
POST /login
↓
Spring Boot API

```

Key Features of Angular

1. TypeScript Based

Angular uses TypeScript instead of plain JavaScript.

```

TypeScript
let name: string = "Lawrence";

```

Benefits:

- Type Safety
- Better IDE Support
- Compile-time Error Detection

2. Component-Based Architecture

Everything is a component.

```

App Component
├── Header Component
├── Sidebar Component
├── Dashboard Component
└── Footer Component

```

Example:

```

TypeScript
@Component({
  selector: 'app-user'
})
export class UserComponent{}

```


Benefits:

- Reusability
 - Maintainability
 - Easy Testing
-

3. Dependency Injection (DI)

Angular automatically creates and injects objects.

Without DI:

```
TypeScript
let service = new UserService();
```

With DI:

```
TypeScript
constructor(private userService: UserService){}
```

Benefits:

- Loose Coupling
 - Easy Unit Testing
-

4. Two-Way Data Binding

Synchronizes UI and Component data.

```
HTML
<input [(ngModel)]="name">
```

```
TypeScript
name = "Lawrence";
```

If UI changes → variable changes.

If variable changes → UI changes.

5. One-Way Data Binding

```
HTML
{{username}}
```

Displays component data in HTML.

6. Directives

Used to manipulate DOM.

ngIf

```
HTML
<div *ngIf="isLoggedIn">
  Welcome
</div>
```

ngFor

```
HTML
<li *ngFor="let user of users">
  {{user.name}}
</li>
```

ngClass

```
HTML
<div [ngClass]="{'active': true}">
```

7. Routing

Allows navigation between pages.

```
TypeScript
const routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent }
];
```

URL:

```
/home
/about
```

8. Services

Business logic should be placed in services.

```
TypeScript
@Injectable()
export class UserService {
}
```

Example:

- API Calls
- Data Processing
- Shared Logic

9. HTTP Client

Calling Backend APIs.

```
TypeScript
this.http.get('/api/users')
```

POST Request:

```
TypeScript
this.http.post('/api/user', payload)
```

Returns Observable.

10. RxJS & Observables

Angular heavily uses Observables.

```
TypeScript
this.userService getUsers()
  .subscribe(data=>{
    console.log(data);
  });
```

Benefits:

- Async Programming
- Event Handling
- Stream Processing

11. Forms

Template Driven Forms

```
HTML
<input [(ngModel)]="username">
```

Simple forms.

Reactive Forms

TypeScript

```
this.form = new FormGroup({
  username: new FormControl('')
});
```

Preferred in enterprise applications.

12. Pipes

Used for data formatting.

HTML

```
{{today | date}}
```

HTML

```
{{salary | currency}}
```

Output:

```
₹50,000
```

13. Lazy Loading

Loads modules only when needed.

TypeScript

```
{
  path: 'admin',
  loadChildren: () => import('./admin.module')
}
```

Benefits:

- Faster Initial Load
 - Better Performance
-

14. Angular CLI

Command line tool.

Create Project:

Bash

```
ng new project-name
```

Create Component:

```
Bash
ng generate component user
```

Run Project:

```
Bash
ng serve
```

Angular Lifecycle Hooks

Most asked interview question.

ngOnInit()

Called after component initialization.

```
TypeScript
ngOnInit(){
  this.loadUsers();
}
```

ngOnChanges()

Called when Input changes.

```
TypeScript
ngOnChanges(){
}
```

ngOnDestroy()

Cleanup subscriptions.

```
TypeScript
ngOnDestroy(){
  this.subscription.unsubscribe();
}
```

Angular Version Evolution

Version	Important Features
AngularJS (1.x)	JavaScript, MVC
Angular 2	Complete Rewrite, TypeScript
Angular 4-8	Performance Improvements
Angular 9	Ivy Renderer
Angular 12	Faster Build
Angular 14	Typed Forms

Version	Important Features
Angular 15	Standalone Components
Angular 16	Signals, Better Reactivity
Angular 17+	New Control Flow, Improved Performance

Angular vs React

Feature	Angular	React
Type	Framework	Library
Developed By	Google	Meta
Language	TypeScript	JavaScript/TypeScript
Learning Curve	Steeper	Easier
Routing	Built-in	External Library
Dependency Injection	Yes	No
State Management	Built-in Support	Redux/Zustand etc.
DOM	Real DOM + Change Detection	Virtual DOM
Data Binding	One-way & Two-way	One-way
Enterprise Apps	Excellent	Good
Flexibility	Less	More
Boilerplate	More	Less

Interview Answer: Angular vs React

If asked in an interview:

Angular is a complete framework developed by Google that provides routing, dependency injection, forms, HTTP client, and state management out of the box. React is a UI library developed by Meta that focuses only on building user interfaces and relies on external libraries for routing and state management. Angular uses TypeScript by default and supports two-way data binding, while React primarily uses one-way data flow with a Virtual DOM.

Angular Topics You Should Know for a Full-Stack Interview

Since you already know API calling and payload passing, focus on:

1. Components
2. Data Binding
3. Directives (**ngIf**, **ngFor**)
4. Services

5. Dependency Injection
6. Routing
7. Reactive Forms
8. Observables & RxJS
9. HTTP Interceptor
10. Guards
11. Lazy Loading
12. Lifecycle Hooks
13. Component Communication
14. JWT Authentication Flow
15. State Management Basics (NgRx/Signals)
16. Angular 16+ Signals
17. Change Detection

These are the areas most commonly asked in Angular full-stack developer interviews.

What are components in angular interview question

What is a Component in Angular?

A **Component** is the basic building block of an Angular application.

A component controls:

- The **HTML View (UI)**
- The **Business Logic**
- The **Data displayed on the page**

Every screen or reusable part of a screen is usually a component.

Example

```

Login Page
├── Header Component
├── Login Component
└── Footer Component
  
```

Each part is an independent component.

Component Structure

When you create a component:

```
Bash
ng generate component user
```

Angular creates:

```
user.component.ts
user.component.html
user.component.css
user.component.spec.ts
```

TypeScript File

```
TypeScript
import { Component } from '@angular/core';

@Component({
  selector: 'app-user',
  templateUrl: './user.component.html',
  styleUrls: ['./user.component.css']
})
export class UserComponent {
  username = 'Lawrence';
}
```

HTML File

```
HTML
<h1>{{username}}</h1>
```

Output:

```
Lawrence
```

Important Parts of @Component

```
TypeScript
@Component({
  selector: 'app-user',
  templateUrl: './user.component.html',
  styleUrls: ['./user.component.css']
})
```

selector

HTML tag used to render component.

```
HTML
<app-user></app-user>
```

templateUrl

Points to HTML file.

```
TypeScript
```



```
templateUrl: './user.component.html'
```

styleUrls

Points to CSS file.

```
TypeScript
styleUrls: ['./user.component.css']
```

Why Do We Use Components?

Without Components:

```
One huge HTML page
5000+ lines
Difficult to maintain
```

With Components:

```
Dashboard
├── Header
├── Sidebar
├── User List
└── Footer
```

Benefits:

- Reusable
- Maintainable
- Testable
- Modular

Component Lifecycle

Angular creates and destroys components during application execution.

Common Lifecycle Hooks:

ngOnInit()

Runs once after component creation.

```
TypeScript
ngOnInit() {
  this.loadUsers();
}
```

Interview Question

Q: Why use ngOnInit instead of constructor?

Answer:

- Constructor is for dependency injection.
- `ngOnInit` is for initialization logic and API calls.

ngOnChanges()

Runs when an `@Input()` value changes.

```
TypeScript
@Input() userId!: number;

ngOnChanges() {
  console.log('Input changed');
}
```

ngOnDestroy()

Runs before component is removed.

```
TypeScript
ngOnDestroy() {
  this.subscription.unsubscribe();
}
```

Used to:

- Remove subscriptions
- Clear timers
- Prevent memory leaks

Component Communication

A very common interview topic.

Parent → Child

Using `@Input()`

Parent

```
HTML
<app-child [name]="username"></app-child>
```

Child

```
TypeScript
@Input() name!: string;
```

Data flows from Parent to Child.

Child → Parent

Using `@Output()`

Child

```
TypeScript
@Output() save = new EventEmitter();

saveUser() {
  this.save.emit();
}
```

Parent

```
HTML
<app-child (save)="saveData()"></app-child>
```

Data/event flows from Child to Parent.

Real-Time Example

Suppose you're building an Employee Management System.

```
App Component
├── Header Component
├── Sidebar Component
├── Employee List Component
├── Employee Details Component
└── Footer Component
```

Employee List Component

Displays employee records.

Employee Details Component

Displays selected employee details.

This separation makes maintenance easier.

Frequently Asked Component Interview Questions

Q1. What is a Component?

Answer:

A Component is the fundamental building block of Angular applications. It controls a portion of the UI and contains HTML, CSS, and TypeScript logic.

Q2. What is the difference between Component and Service?

Component	Service
Handles UI	Handles business logic
Has HTML template	No UI
Created using @Component	Created using @Injectable
Used for displaying data	Used for API calls, calculations

Example:

```

UserComponent
  ↓
UserService
  ↓
Backend API

```

Q3. Can Components Communicate?

Yes.

- Parent → Child using @Input()
- Child → Parent using @Output()
- Unrelated Components using Services

Q4. What is the Difference Between Constructor and ngOnInit?

Constructor	ngOnInit
Dependency Injection	Initialization Logic
Called first	Called after component creation
Avoid API calls here	API calls preferred here

Q5. What Happens When a Component Loads?

```

Component Created
  ↓
Constructor Called
  ↓
ngOnChanges()
  ↓
ngOnInit()
  ↓
View Rendered

```

Q6. What is a Standalone Component? (Angular 15+)

Instead of declaring components in a module:

```

TypeScript
@Component({
  standalone: true
})
export class UserComponent {}

```

Benefits:

- Less boilerplate
- Simpler architecture
- Faster development

This is a popular Angular 15+ interview question.

Interview Answer (2-Minute Version)

A Component is the basic building block of an Angular application. It consists of a TypeScript class, an HTML template, and CSS styles. Components are responsible for controlling a part of the UI and handling user interactions. Angular applications are built using multiple reusable components. Components can communicate using `@Input()` and `@Output()`, and they follow lifecycle hooks such as `ngOnInit`, `ngOnChanges`, and `ngOnDestroy`. Components focus on UI, while services handle business logic and API calls.

Explain Single Page Application and Multi page application

Single Page Application (SPA) vs Multi Page Application (MPA)

This is one of the most common Angular and React interview questions.

What is a Single Page Application (SPA)?

A **Single Page Application** loads the entire application **once**. After that, only the required data is fetched from the backend via APIs without reloading the whole page.

Frameworks:

- Angular
- React
- Vue

How SPA Works

```

User Opens Website
  ↓
Browser Downloads
HTML + CSS + JS
  ↓
Angular App Loads
  ↓

```

```
User Clicks Dashboard
↓
API Call
↓
Only Data Changes
(No Full Page Reload)
```

Example

Suppose you are using Gmail.

```
Inbox
↓
Sent Items
↓
Drafts
```

Notice the page doesn't refresh completely. Only the content changes.

This is SPA behavior.

SPA Architecture

```
Browser
↓
Angular Application
↓
REST API
(Spring Boot)
↓
Database
```

Example:

```
TypeScript
this.http.get('/api/employees')
```

Backend returns:

```
JSON
[
  {
    "id": 1,
    "name": "John"
  }
]
```

Angular updates the UI without reloading the page.

Advantages of SPA

Faster Navigation

Page resources load once.

```
Home → Dashboard → Profile
```

No full page reload.

Better User Experience

Feels like a desktop/mobile app.

Examples:

- Gmail
- Google Maps
- Netflix
- Facebook

Reduced Server Load

Only API calls are made.

```
http
GET /employees
```

Instead of returning a full HTML page.

Frontend and Backend Separation

```
Angular
  ↓
REST API
  ↓
Spring Boot
```

Easy to scale and maintain.

Disadvantages of SPA

Initial Loading Time

First load may be larger.

```
main.js
vendor.js
styles.css
```

Need to download JavaScript bundles.

SEO Challenges

Search engines may struggle with JavaScript-rendered content unless server-side rendering is used.

What is a Multi Page Application (MPA)?

In an MPA, every request loads a completely new page from the server.

Traditional Technologies:

- JSP
- Servlet
- PHP
- ASP.NET MVC
- Thymeleaf

How MPA Works

```

User Clicks Dashboard
      ↓
Browser Sends Request
      ↓
Server Generates HTML
      ↓
Entire Page Reloads
  
```

Every navigation causes a full page refresh.

MPA Architecture

```

Browser
  ↓
Server
(JSP/Thymeleaf)
  ↓
Database
  
```

Server generates HTML for every request.

Example

User visits:

```
/login
```

Server returns:


```
HTML
<html>
Login Page
</html>
```

User clicks Dashboard:

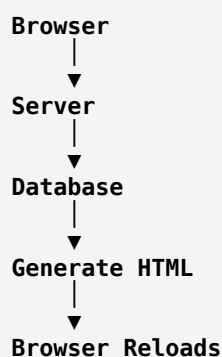
```
/dashboard
```

Server returns:

```
HTML
<html>
Dashboard Page
</html>
```

Entire page reloads.

MPA Flow Diagram



Real Example

SPA (Angular)

```
Home
↓
Dashboard
↓
Profile
```

Browser URL changes:

```
/home
/dashboard
/profile
```

But only content changes.

No full refresh.

MPA (JSP)

```

Home
↓
Request Sent
↓
Server Generates Page
↓
Browser Reloads

Dashboard
↓
Request Sent
↓
Server Generates Page
↓
Browser Reloads

```

Angular Routing in SPA

```

TypeScript
const routes = [
  { path: 'home', component: HomeComponent },
  { path: 'profile', component: ProfileComponent }
];

```

Navigation:

```

TypeScript
this.router.navigate(['/profile']);

```

No page reload.

SPA vs MPA

Feature	SPA	MPA
Page Reload	No	Yes
Speed	Faster after first load	Slower
User Experience	Better	Traditional
Frontend	Angular/React/Vue	JSP/PHP/Thymeleaf
Backend	REST APIs	Generates HTML
SEO	More complex	Easier
Server Load	Lower	Higher
Scalability	Better	Harder
Mobile-like Experience	Yes	No

Interview Question

Q: Why is Angular called a Single Page Application framework?

Answer:

Angular applications load a single HTML page initially. After that, navigation between screens happens through Angular Routing, and data is fetched using API calls. The browser does not reload the entire page; only the required content is updated dynamically, which makes Angular a Single Page Application framework.

Interview Question

Q: What happens when you click a menu item in Angular?

Answer:

1. User clicks menu item.
2. Angular Router intercepts the request.
3. Corresponding Component is loaded.
4. API call is made if required.
5. Backend returns JSON data.
6. Angular updates the DOM.
7. No full page refresh occurs.

```

Click Menu
  ↓
Router
  ↓
Component
  ↓
API Call
  ↓
Backend
  ↓
JSON Response
  ↓
UI Updated
  
```

This flow is often asked in Angular full-stack interviews because it demonstrates understanding of SPA architecture.

For an **Angular Full Stack Developer interview**, you don't need to master all of TypeScript, but you should be comfortable with the concepts Angular uses daily.

TypeScript Topics You Must Know

1. What is TypeScript?

TypeScript is a superset of JavaScript developed by Microsoft.

It adds:

- Static Typing
- Interfaces
- Classes
- Generics
- Decorators
- Better Tooling

Example:

JavaScript

```
JavaScript
let age = "25";
```

No error.

TypeScript

```
TypeScript
let age: number = 25;
```

Compile-time type checking.

2. Data Types

```
TypeScript
let name: string = "Lawrence";
let age: number = 25;
let active: boolean = true;
```

Arrays

```
TypeScript
let users: string[] = ["John", "Mike"];
```

Tuple

```
TypeScript
let employee: [number, string];

employee = [1, "Lawrence"];
```

3. Type Inference

TypeScript can infer types.

```
TypeScript
```

```
let name = "Lawrence";
```

TypeScript automatically treats it as:

```
TypeScript
string
```

4. any vs unknown

any

```
TypeScript
let data: any;

data = 10;
data = "Hello";
```

No type checking.

unknown

```
TypeScript
let data: unknown;
```

Safer than **any**.

Interview Question

any vs unknown?

Answer:

any disables type checking completely. **unknown** requires type checking before use and is safer.

5. Functions

```
TypeScript
function add(
  a:number,
  b:number
):number{

  return a+b;
}
```

Optional Parameters

```
TypeScript
function greet(
  name?:string
){}
```

Default Parameters

```
TypeScript
function greet(
  name:string="Guest"
){}
```

6. Interfaces

Very important for Angular.

Used to define object structure.

```
TypeScript
interface Employee {
  id:number;
  name:string;
  salary:number;
}
```

Usage:

```
TypeScript
let emp: Employee = {
  id:1,
  name:"Lawrence",
  salary:50000
};
```

Angular Example

```
TypeScript
export interface User {
  id:number;
  name:string;
  email:string;
}
```

API response mapping.

Interview Question

Interface vs Type?

Most Angular projects use:

```
TypeScript
interface
```

for models.

7. Classes

Angular components and services are classes.

```
TypeScript
class Employee {

  id:number;
  name:string;

  constructor(
    id:number,
    name:string
  ){

    this.id=id;
    this.name=name;

  }
}
```

8. Access Modifiers

Public

```
TypeScript
public name:string;
```

Accessible everywhere.

Private

```
TypeScript
private password:string;
```

Accessible only inside class.

Protected

```
TypeScript
protected salary:number;
```

Accessible in child classes.

Interview Question

private vs protected?

private	protected
Current Class Only	Current + Child Classes

9. Inheritance

```
TypeScript
class Person {
  name:string="John";
}

class Employee
extends Person {
  salary:number=50000;
}
```

10. Abstract Class

```
TypeScript
abstract class Shape {
  abstract area():number;
}
```

Child must implement method.

11. Generics

Frequently asked.

Without Generics:

```
TypeScript
function getData(
  data:any
){
```



```
return data;
}
```

With Generics:

```
TypeScript
function getData<T>(
  data:T
):T{

  return data;
}
```

Usage:

```
TypeScript
getData<string>(
  "Hello"
);
```

Angular Example

```
TypeScript
this.http
.get<User[]>(
  '/users'
);
```

12. Enums

```
TypeScript
enum Role {

  ADMIN,
  USER,
  GUEST

}
```

Usage:

```
TypeScript
Role.ADMIN
```

13. Union Types

```
TypeScript
let id:
  number | string;
```

Valid:

```
TypeScript
id = 1;
id = "A101";
```

14. Type Aliases

```
TypeScript
type UserId =
  number | string;
```

15. Type Assertion

```
TypeScript
let data:any="Hello";

let name =
  data as string;
```

16. Arrow Functions

Very common in Angular.

```
TypeScript
const add = (
  a:number,
  b:number
) => a+b;
```

17. Spread Operator

```
TypeScript
const arr1 =
  [1,2];

const arr2 =
  [...arr1,3,4];
```

Output:

```
TypeScript
[1,2,3,4]
```

18. Destructuring

```
TypeScript
const user = {
  name:"Lawrence",
  age:25
};

const {
```

```
name,  
age  
} = user;
```

19. Promise

```
TypeScript  
const promise =  
  new Promise(  
    resolve => {  
  
      resolve("Success");  
  
    });
```

20. Async/Await

```
TypeScript  
async function getUsers(){  
  
  const users =  
  await fetch('/users');  
  
}
```

21. Decorators

Extremely important for Angular.

```
TypeScript  
@Component()  
@Injectable()  
@Directive()  
@Pipe()
```

Decorators add metadata.

22. Modules

```
TypeScript  
export class User {}
```

Import:

```
TypeScript  
import {  
  User  
}  
from './user';
```

23. Optional Chaining

Very common.

```
TypeScript
user?.address?.city
```

Prevents:

```
Cannot read property...
```

errors.

24. Nullish Coalescing

```
TypeScript
let name =
  username ?? "Guest";
```

25. Readonly

```
TypeScript
readonly id:number=1;
```

Cannot be modified.

TypeScript Concepts Most Asked in Angular Interviews

Beginner

- Data Types
- Functions
- Arrays
- Interfaces
- Classes
- Access Modifiers

Intermediate

- Generics
- Enums
- Type Aliases
- Union Types
- Arrow Functions
- Modules

Advanced

- Decorators
- Abstract Classes
- Optional Chaining
- Async/Await
- Generics with API Responses

Top 10 Interview Questions

Q1. What is TypeScript?

TypeScript is a statically typed superset of JavaScript that adds type safety and object-oriented features.

Q2. Why Angular uses TypeScript?

- Type Safety
- Better Tooling
- OOP Features
- Decorators
- Easier Maintenance

Q3. Interface vs Class?

Interface	Class
Defines Structure	Defines Structure + Implementation
No Object Creation	Object Creation Possible

Q4. any vs unknown?

`unknown` is safer because type checking is required before use.

Q5. What are Generics?

Generics allow writing reusable and type-safe code.

```
TypeScript
function getData<T>(  
  data:T  
) :T
```

Q6. public vs private vs protected?

Modifier	Access
public	Everywhere
private	Same Class
protected	Class + Child

Q7. What are Decorators?

Special annotations that provide metadata.

Examples:

```
TypeScript
@Component
@Injectable
```

Q8. What is Optional Chaining?

```
TypeScript
user?.address?.city
```

Safely accesses nested properties.

Q9. What is a Union Type?

```
TypeScript
number | string
```

Variable can hold multiple types.

Q10. Why use Interfaces in Angular?

To define API response models and ensure type safety.

```
TypeScript
interface User {
  id: number;
  name: string;
}
```

For your Angular interviews, the **highest-priority TypeScript topics** are:

1. Interfaces
2. Classes

3. Access Modifiers
4. Generics
5. Decorators
6. Arrow Functions
7. Async/Await
8. Optional Chaining
9. Union Types
10. Modules (import/export)

These appear in Angular interviews far more often than advanced TypeScript-only topics.

JavaScript should know concepts

For **Angular Full Stack Developer interviews**, JavaScript fundamentals are extremely important because Angular itself is built on JavaScript/TypeScript concepts.

JavaScript Concepts You Must Know

1. var vs let vs const

var

```
JavaScript
var name = "Lawrence";
```

Problems:

- Function scoped
- Can be redeclared
- Hoisted

let

```
JavaScript
let age = 25;
```

- Block scoped
- Can be reassigned
- Cannot be redeclared

const

```
JavaScript
const PI = 3.14;
```

- Block scoped
- Cannot be reassigned

Interview Question

var vs let vs const

Feature	var	let	const
Scope	Function	Block	Block
Redeclare	Yes	No	No
Reassign	Yes	Yes	No
Hoisted	Yes	Yes	Yes

2. Scope

Global Scope

```
JavaScript
let name = "Lawrence";
```

Accessible everywhere.

Function Scope

```
JavaScript
function test() {
  var age = 25;
}
```

Only inside function.

Block Scope

```
JavaScript
if(true){
  let city = "Mumbai";
}
```

Only inside block.

3. Hoisting

JavaScript moves declarations to the top.

```
JavaScript
console.log(a);
var a = 10;
```

Output:

```
JavaScript
undefined
```

Equivalent:

```
JavaScript
var a;
console.log(a);
a = 10;
```

Interview Question

What is Hoisting?

JavaScript's behavior of moving declarations to the top of their scope before execution.

4. == vs ===

== (Loose Equality)

```
JavaScript
5 == "5"
```

Output:

```
JavaScript
true
```

Type conversion occurs.

=== (Strict Equality)

```
JavaScript
5 === "5"
```

Output:

```
JavaScript
false
```

Checks:

- Value
- Type

Always prefer:

```
JavaScript
===
```

5. Functions

Normal Function

```
JavaScript
function add(a,b){
  return a+b;
}
```

Function Expression

```
JavaScript
const add =
  function(a,b){
    return a+b;
  };
```

Arrow Function

```
JavaScript
const add =
  (a,b) => a+b;
```

Very common in Angular.

6. Arrow Function vs Normal Function

Normal Function

Has its own:

```
JavaScript
this
```

Arrow Function

Inherits **this** from parent.

Example:

```
JavaScript
class User {
  name = "Lawrence";

  show = () => {
    console.log(this.name);
  }
}
```

Interview Question

Why use Arrow Functions?

To preserve lexical **this**.

7. this Keyword

Refers to the current execution context.

Example:

```
JavaScript
const user = {
  name: "Lawrence",

  show(){
    console.log(this.name);
  }
}
```

Output:

```
JavaScript
Lawrence
```

8. Closures

Most asked JavaScript interview question.

A Closure is a function that remembers variables from its outer scope even after the outer function has finished executing.

Example:

```
JavaScript
function outer(){
```

```
let count = 0;

return function(){

    count++;

    console.log(count);

};

}

const increment =
    outer();

increment();
increment();
```

Output:

```
JavaScript
1
2
```

Uses

- Data Privacy
- Counters
- Memoization

9. Callback Functions

Function passed as argument.

```
JavaScript
function greet(callback){

    callback();

}

greet(() => {

    console.log("Hello");

});
```

10. Promise

Handles asynchronous operations.

Create Promise

```
JavaScript
```

```
const promise =
  new Promise(
    (resolve, reject) => {
      resolve("Success");
    }
  );
```

Consume Promise

```
JavaScript
promise.then(result => {
  console.log(result);
});
```

Output:

```
JavaScript
Success
```

Promise States

```
Pending
↓
Resolved
or
Rejected
```

11. Async/Await

Modern way of handling promises.

```
JavaScript
async function getData(){
  const data =
    await fetch('/users');
}
```

Cleaner than `.then()`.

Interview Question

Promise vs Async/Await

Async/Await is syntactic sugar over Promises and provides cleaner code.

12. Event Loop

Very important.

JavaScript is:

Single Threaded

Yet handles async operations.

Example

```
JavaScript
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output:

```
A
C
B
```

Because:

```
Call Stack
  ↓
Web APIs
  ↓
Callback Queue
  ↓
Event Loop
```

13. Call Stack

Tracks function execution.

```
JavaScript
function a(){
  b();
}

function b(){
  c();
}

function c() {}
```

Stack:

```
c
b
a
```

14. Spread Operator (...)

```
JavaScript
const arr1 =
  [1,2];

const arr2 =
  [...arr1,3];
```

Output:

```
JavaScript
[1,2,3]
```

Object Example

```
JavaScript
const user = {
  name:"Lawrence"
};

const employee = {
  ...user,
  salary:50000
};
```

15. Rest Operator

```
JavaScript
function sum(...numbers){
  return numbers.reduce(
    (a,b)=>a+b
  );
}
```

16. Destructuring

Object

```
JavaScript
const user = {
  name:"Lawrence",
  age:25
};

const {
  name,
```

```
age  
} = user;
```

Array

```
JavaScript  
const nums =  
  [1,2,3];  
  
const [a,b] =  
  nums;
```

17. Array Methods

Frequently asked.

map()

Transform data.

```
JavaScript  
const nums =  
  [1,2,3];  
  
const result =  
  nums.map(  
    n => n*2  
  );
```

Output:

```
JavaScript  
[2,4,6]
```

filter()

Filter data.

```
JavaScript  
const nums =  
  [1,2,3,4];  
  
const result =  
  nums.filter(  
    n => n>2  
  );
```

Output:

```
JavaScript  
[3,4]
```


find()

```
JavaScript
users.find(
  user => user.id===1
);
```

Returns first match.

reduce()

```
JavaScript
const sum =
  nums.reduce(
    (a,b)=>a+b,
    0
  );
```

Output:

```
JavaScript
10
```

forEach()

```
JavaScript
users.forEach(
  user =>
    console.log(user)
);
```

18. map vs forEach

map	forEach
Returns New Array	Returns Nothing
Used for Transformation	Used for Iteration

19. Null vs Undefined

Undefined

```
JavaScript
let name;
```

Output:

```
JavaScript
undefined
```

Null

```
JavaScript
let name = null;
```

Explicitly assigned empty value.

20. Optional Chaining

```
JavaScript
user?.address?.city
```

Avoids:

```
JavaScript
Cannot read property...
```

errors.

21. Template Literals

```
JavaScript
const name =
  "Lawrence";

console.log(
  `Hello ${name}`
);
```

Output:

```
JavaScript
Hello Lawrence
```

22. DOM Manipulation

Basic knowledge is often asked.

```
JavaScript
document.getElementById("name");
```

```
JavaScript
document.querySelector(".btn");
```

Top JavaScript Interview Questions for Angular Developers

Q1. Difference between let, var, and const?

Q2. What is Hoisting?

Q3. Difference between == and ===?

Q4. What is Closure?

Q5. What is Event Loop?

Q6. What is a Promise?

Q7. Difference between Promise and Async/Await?

Q8. Difference between map(), filter(), reduce(), and forEach()?

Q9. What is the difference between null and undefined?

Q10. Why are Arrow Functions used?

Must-Know JavaScript Topics for Angular Interviews

Essential

- ✓ let, var, const
- ✓ Scope
- ✓ Hoisting
- ✓ == vs ===
- ✓ Functions & Arrow Functions
- ✓ this keyword
- ✓ Closures
- ✓ Callbacks
- ✓ Promises
- ✓ Async/Await
- ✓ Event Loop

Frequently Asked

- ✓ map()
- ✓ filter()
- ✓ reduce()
- ✓ forEach()
- ✓ Spread Operator
- ✓ Destructuring
- ✓ Optional Chaining

Advanced (Good to Know)

- ✓ Call Stack
- ✓ Memory Management
- ✓ Prototype & Prototype Chain
- ✓ Debouncing & Throttling
- ✓ Currying

If you're preparing for Angular + Java Full Stack interviews, the **top 10 most frequently asked JavaScript topics** are:

1. Closures
2. Event Loop
3. Promise
4. Async/Await
5. Arrow Functions
6. **this**
7. **var** vs **let** vs **const**
8. Hoisting
9. **map/filter/reduce**
10. **==** vs **===**

React should know concepts

If you're interviewing for **Angular Full Stack Developer** roles, React is usually not mandatory. However, many companies ask basic React questions to evaluate frontend knowledge.

React Concepts You Should Know

What is React?

React is a JavaScript library developed by Meta for building user interfaces.

It follows a **Component-Based Architecture**.

Example:

```

App
├── Header
├── Sidebar
├── Dashboard
└── Footer
  
```

React Features

- Component-Based
- Virtual DOM
- JSX
- One-Way Data Binding
- Reusable Components
- Fast Rendering
- Rich Ecosystem

Angular vs React

Angular	React
Framework	Library
TypeScript by default	JavaScript/TypeScript
Two-way binding	One-way binding
Built-in routing	React Router
Built-in DI	No DI
Opinionated	Flexible
Enterprise-focused	Flexible ecosystem

JSX

JSX = JavaScript XML

Allows HTML-like syntax inside JavaScript.

```

JavaScript
function App() {
  return (
    <h1>Hello React</h1>
  );
}
  
```

React converts JSX to JavaScript.

Interview Question

What is JSX?

JSX is a syntax extension that allows writing HTML-like code inside JavaScript. It improves readability and is compiled into React element creation code.

Components

Components are reusable UI blocks.

Functional Component

Modern React uses functional components.

```
JavaScript
function Welcome() {
  return <h1>Hello</h1>;
}
```

Component Usage

```
JavaScript
<Welcome />
```

Props

Props are used to pass data from Parent to Child.

Parent

```
JavaScript
<User name="Lawrence" />
```

Child

```
JavaScript
function User(props) {
  return <h1>{props.name}</h1>;
}
```

Output:

```
Lawrence
```

React vs Angular

Angular	React
@Input()	Props
@Output()	Callback Function

State

State stores component data.

Example

```
JavaScript
const [count, setCount] =
  useState(0);
```

Value:

```
JavaScript
count
```

Update:

```
JavaScript
setCount(1);
```

Why State?

When state changes:

Component Re-renders

useState Hook

Most commonly asked React interview topic.

```
JavaScript
import { useState }
from 'react';

function Counter() {

  const [count, setCount]
= useState(0);

  return (
    <>
```

```
<p>{count}</p>

<button
  onClick={() =>
    setCount(count+1)
  }>
  Increment
</button>
</>
);
}
```

Interview Question

What is useState?

A React Hook used to create and manage component state in functional components.

Event Handling

```
JavaScript
<button
  onClick={handleClick}>
  Click
</button>
```

```
JavaScript
const handleClick = () => {
  console.log("Clicked");
};
```

Conditional Rendering

Using Ternary

```
JavaScript
{
  isLoggedIn
  ? <Home />
  : <Login />
}
```

Using &&

```
JavaScript
{
  isAdmin &&
```



```
<AdminPanel />
}
```

List Rendering

Equivalent of Angular's `*ngFor`.

```
JavaScript
const users =
  ["John", "Mike"];
```

```
JavaScript
<ul>

{
  users.map(user => (
    <li>
      {user}
    </li>

  ))
}

</ul>
```

Keys

```
JavaScript
users.map(user => (
  <li key={user.id}>
    {user.name}
  </li>

))
```

Keys help React identify changed elements efficiently.

useEffect Hook

Second most asked React topic.

Used for:

- API Calls
- Lifecycle Events
- Side Effects

Example

```
JavaScript
```

```
import {
  useEffect
} from 'react';

useEffect(() => {

  console.log(
    "Component Loaded"
  );

}, []);
```

Runs once.

API Call Example

```
JavaScript
useEffect(() => {

  fetch('/users')
  .then(res => res.json())
  .then(data => {

  });

}, []);
```

React Lifecycle vs Angular Lifecycle

Angular	React
ngOnInit	useEffect([])
ngOnDestroy	cleanup function
ngOnChanges	useEffect([dependency])

Cleanup Function

Equivalent of `ngOnDestroy`.

```
JavaScript
useEffect(() => {

  return () => {

    console.log(
      "Component Destroyed"
    );

  };

}, []);
```

useRef

Used to access DOM elements.

```
JavaScript
const inputRef =
  useRef();
```

```
JavaScript
<input
  ref={inputRef}
/>
```

Forms

Controlled Components.

```
JavaScript
const [name, setName]
= useState('');
```

```
JavaScript
<input
  value={name}
  onChange={(e)=>
    setName(
      e.target.value
    )
  }
/>
```

Context API

Used for global state management.

Equivalent to:

- Angular Services
- BehaviorSubject

Create Context

```
JavaScript
const UserContext =
  createContext();
```

Provider

```
JavaScript
```

```
<UserContext.Provider
  value={user}
>

</UserContext.Provider>
```

Consume

```
JavaScript
const user =
  useContext(
    UserContext
  );
```

Virtual DOM

Very common interview question.

Traditional DOM

```
DOM Change
  ↓
Entire DOM Update
```

Slow.

Virtual DOM

```
State Change
  ↓
Virtual DOM
  ↓
Diffing
  ↓
Update Changed Parts Only
```

Fast.

Interview Question

What is Virtual DOM?

Virtual DOM is a lightweight copy of the real DOM. React compares changes using a diffing algorithm and updates only the affected parts of the UI.

React Routing

Using React Router.

```
JavaScript
<Route
  path="/home"
  element={<Home />}
/>
```

API Calls

Using Fetch:

```
JavaScript
fetch('/users')
```

Or Axios:

```
JavaScript
axios.get('/users')
```

React Hooks

Most important hooks.

useState

Manage state.

```
JavaScript
useState()
```

useEffect

Lifecycle and side effects.

```
JavaScript
useEffect()
```

useContext

Global state.

```
JavaScript
useContext()
```

useRef

DOM access.

```
JavaScript  
useRef()
```

useMemo

Performance optimization.

```
JavaScript  
useMemo()
```

useCallback

Memoize functions.

```
JavaScript  
useCallback()
```

Most Asked React Interview Questions

Q1. What is React?

A JavaScript library for building UI components.

Q2. What is JSX?

HTML-like syntax inside JavaScript.

Q3. What is a Component?

Reusable UI building block.

Q4. What are Props?

Data passed from Parent to Child.

Q5. What is State?

Component-specific data that triggers re-rendering when updated.

Q6. What is useState?

Hook used to manage state.

Q7. What is useEffect?

Hook used for side effects such as API calls and lifecycle handling.

Q8. What is Virtual DOM?

A lightweight DOM copy used to efficiently update only changed UI parts.

Q9. Difference Between State and Props?

Props	State
Read Only	Mutable
Parent → Child	Internal Data
External Input	Component Data

Q10. React vs Angular?

React	Angular
Library	Framework
JSX	HTML Templates
useState	Component Properties
useEffect	Lifecycle Hooks
Props	@Input
Context/Redux	Services + RxJS

React Topics to Know for Interviews

Essential

- ✓ Components
- ✓ JSX
- ✓ Props
- ✓ State
- ✓ useState
- ✓ useEffect
- ✓ Event Handling

- ✓ Conditional Rendering
- ✓ List Rendering

Intermediate

- ✓ Context API
- ✓ React Router
- ✓ Forms
- ✓ API Calls
- ✓ Virtual DOM

Advanced

- ✓ useMemo
- ✓ useCallback
- ✓ Redux (basic knowledge)
- ✓ Performance Optimization

For a Java + Angular Full Stack developer, the React topics most likely to be asked are:

1. React vs Angular
2. JSX
3. Components
4. Props
5. State
6. useState
7. useEffect
8. Virtual DOM
9. React Router
10. Context API vs Angular Services/RxJS